LAB 4

File	Purpose
main.py	Starts the game
game.py	Handles gameplay and turns
board.py	Handles ships, shots and hits
ai.py	Controls computer targeting

Run the original game:

python main.py

Record the BEFORE video:

Before making changes:

1. Open the terminal.
2. Run the game.
3. Record your screen for 10 seconds.
4. Show the original gameplay and the incorrect behaviour.

before_changes.mp4

Start Vibe Coding with ChatGPT

Now we'll fix the project using ChatGPT.

Open your ChatGPT conversation (this one).

First, upload these four files:

- main.py
- game.py

- board.py
- ai.py

Alternatively, paste their contents here.

I am working on Lab 4: VibeCoding, Scenario 19 – Battleship vs AI.

I have an existing Python Battleship game with four modules: main.py, game.py, board.py and ai.py.

First, inspect the existing implementation carefully. Do not rewrite the entire project.

Identify the coordinate representation mismatch between player input, board ship positions and AI shots.

Fix Task 1: Make coordinate representation consistent throughout the project so that every actual ship coordinate is recognised as a hit and every known empty coordinate is recognised as a miss.

Preserve the existing modular structure and explain:

1. What the original bug was.
2. Which files need modification.
3. What changes you made.
4. How I can test the fix.

Do not implement Tasks 2–4 yet.

Record the AFTER video:

Record another 10-second video showing:

- The corrected gameplay.
- A successful hit or miss.
- AI taking its turn.
- New features working.

after_changes.mp4

Fix the coordinate mismatch

Replace game.py

```
from board import Board
from ai import AI



class Battleship:
    def __init__(self):
        self.player = Board()
        self.enemy = Board()
        self.ai = AI(Board.SIZE)
        self._setup()

    def _setup(self):
        self.player.place_ship({(1, 1), (1, 2), (1, 3)})
        self.player.place_ship({(4, 4), (4, 5)})


        self.enemy.place_ship({(2, 2), (2, 3), (2, 4)})
        self.enemy.place_ship({(4, 0), (5, 0)})

    def show(self):
        print("\nYour shots are coordinates like 2,3 (or q to quit).")
        print("Enemy ship cells remaining:", self.enemy.remaining_cells())

    def run(self):
        print("Battleship")
```

```
while True:
    self.show()
    raw = input("> ").strip().lower()

    if raw == "q":
        print("Game ended.")
        return

    try:
        parts = raw.split(",")

        if len(parts) != 2:
            raise ValueError

        row, col = map(int, parts)
        pos = (row - 1, col - 1)

    except ValueError:
        print("Invalid input. Enter coordinates like 2,3.")
        continue

    if not (
        0 <= pos[0] < Board.SIZE
        and 0 <= pos[1] < Board.SIZE
    ):
        print("Outside board. Choose coordinates from 1 to 6.")
        continue
```

```
if pos in self.enemy.shots:
    print("Already fired there. Choose another cell.")
    continue

hit, sunk_ship = self.enemy.fire(pos)

if hit:
    print("HIT!")
    if sunk_ship is not None:
        print("You sank an enemy ship!")
else:
    print("MISS!")

if self.enemy.all_sunk():
    print("You sank the enemy fleet. You win!")
    return

ai_move = self.ai.choose()

if ai_move is None:
    print("AI has no untried cells remaining.")
    return

try:
    ai_row, ai_col = map(int, ai_move.split(","))
    player_pos = (ai_row - 1, ai_col - 1)
except (ValueError, AttributeError):
```

```
    print("AI generated an invalid move.")
    return

if not (
    0 <= player_pos[0] < Board.SIZE
    and 0 <= player_pos[1] < Board.SIZE
):
    print("AI generated an invalid move.")
    return

print("AI fired at", ai_move)

ai_hit, ai_sunk_ship = self.player.fire(player_pos)
self.ai.record_result(player_pos, ai_hit)

if ai_hit:
    print("AI scored a hit!")
    if ai_sunk_ship is not None:
        print("AI sank one of your ships!")
else:
    print("AI missed.")

if self.player.all_sunk():
    print("The AI sank your fleet. Game over!")
    return
```

Fix Task 1: Consistent Coordinate Handling in my existing Battleship vs AI Python project.

I identified that the AI generates 1-based coordinates, but the Board module stores ship positions using zero-based coordinates.

Inspect the existing code and fix this mismatch without rewriting the entire project.

Requirements:

1. Maintain consistent zero-based coordinates internally in the Board module.
2. Convert player input into zero-based coordinates.
3. Convert AI-generated coordinates correctly before checking the player's board.
4. Ensure all actual ship coordinates are recognised as hits.
5. Ensure known empty coordinates are recognised as misses.
6. Preserve the existing modular structure.

Explain every modification and provide the testing steps. Do not implement Tasks 2, 3 or 4 yet.

Test the following:

Input	Expected result
2,2	HIT
2,3	HIT
2,4	HIT
1,1	MISS

Now implement Task 2: Complete Fleet and Win Logic in my existing Battleship vs AI project.

Inspect the current main.py, game.py, board.py and ai.py before making changes.

Requirements:

1. Support multiple ships on each board.

2. Track damage separately for each individual ship.
3. Detect and report when an individual ship is completely sunk.
4. Prevent repeated shots at previously targeted coordinates.
5. Implement all_sunk() to correctly identify when the entire fleet has been destroyed.
6. End the game immediately when all enemy ships are sunk.
7. Maintain the existing modular structure and keep all game state in memory.
8. Do not introduce CSV, JSON, SQLite or other persistent storage.
9. Do not add unnecessary external dependencies.

Do not rewrite the entire project. Modify only the necessary existing modules.

Explain the changes made in each file and provide test cases for:

- Hitting a ship.
- Missing a ship.
- Sinking one ship.
- Sinking the entire fleet.
- Firing at the same coordinate twice.

Do not implement Tasks 3 and 4 yet.

Action	**Expected output**
Hit a ship cell	HIT!
Hit all cells of one ship	SHIP SUNK!
Fire at an already targeted cell	Already fired there.
Destroy all enemy ships	You sank the fleet!
Game completed	Game ends

Implement Task 3: Improve the AI in my existing Battleship vs AI Python project.

Inspect the existing ai.py, board.py and game.py before making changes.

Requirements:

1. After the AI gets a hit, it should preferentially consider nearby untried cells.
2. Check the four adjacent directions: up, down, left and right.
3. Ensure all candidate coordinates are within the board boundaries.
4. Never allow the AI to shoot the same coordinate twice.
5. If no adjacent untried cells are available, choose another untried cell randomly.
6. Handle the situation where the board has no remaining available cells without crashing.
7. Keep the existing modular structure.
8. Do not add unnecessary external dependencies or persistent storage.

Ensure that the AI receives the actual hit/miss result from its shot so that it can update its targeting strategy.

Do not rewrite the entire project. Do not implement Task 4 yet.

Explain the changes and provide test cases for adjacent targeting, repeated-shot prevention, boundary conditions and a board with no remaining choices.

Situation	**AI behaviour**
AI gets a hit	Prioritises adjacent cells
Adjacent cell already tried	Skips that cell
Adjacent cell outside board	Ignores it
No adjacent cells available	Selects another untried cell
All cells already tried	Handles gracefully without crashing

Implement Task 4: Shot-Level Feedback in my existing Battleship vs AI project.

Inspect the existing main.py, game.py, board.py and ai.py.

Requirements:

1. Ensure HIT, MISS and SUNK feedback is displayed exactly once for each actual shot.
2. Display HIT when a shot hits a ship.
3. Display MISS when a shot does not hit a ship.
4. Display SHIP SUNK when all cells of an individual ship have been hit.
5. Display fleet destroyed and end the game when all ships are sunk.
6. Apply the same feedback rules to both player and AI shots.
7. Internal AI candidate selection must never generate false HIT or MISS messages.
8. Invalid coordinates and repeated shots must not be treated as actual shots.
9. Preserve the existing modular structure and avoid unnecessary code duplication.

Do not rewrite the entire project. Explain the modifications and provide test cases to verify that feedback appears exactly once per actual shot.

Action	Expected feedback
Player hits ship	HIT!
Player misses	MISS!
Player sinks ship	SHIP SUNK!
AI hits ship	AI scored a hit.
AI misses	AI missed.
AI sinks ship	AI sank a ship!

Record the AFTER video:

record another 10-second gameplay video demonstrating the corrected game and new features.

Save it as:

after_changes.mp4